

Camino mínimo en grafos (II)

Técnicas de Diseño de Algoritmos (Ex Algoritmos y
Estructuras de Datos III)

Segundo cuatrimestre 2025

Algoritmos matriciales

Sea $G = (\{1, \dots, n\}, X)$ un digrafo y $\ell : X \rightarrow R$ una función de longitud/peso para las aristas de G . Definimos las siguientes matrices:

Algoritmos matriciales

Sea $G = (\{1, \dots, n\}, X)$ un digrafo y $\ell : X \rightarrow R$ una función de longitud/peso para las aristas de G . Definimos las siguientes matrices:

- $L \in R^{n \times n}$, donde los elementos ℓ_{ij} de L se definen como:

$$\ell_{ij} = \begin{cases} 0 & \text{si } i = j \\ \ell(i \rightarrow j) & \text{si } i \rightarrow j \in X \\ \infty & \text{si } i \rightarrow j \notin X \end{cases}$$

Algoritmos matriciales

Sea $G = (\{1, \dots, n\}, X)$ un digrafo y $\ell : X \rightarrow R$ una función de longitud/peso para las aristas de G . Definimos las siguientes matrices:

- $L \in R^{n \times n}$, donde los elementos ℓ_{ij} de L se definen como:

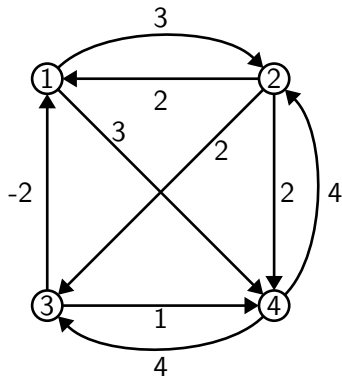
$$\ell_{ij} = \begin{cases} 0 & \text{si } i = j \\ \ell(i \rightarrow j) & \text{si } i \rightarrow j \in X \\ \infty & \text{si } i \rightarrow j \notin X \end{cases}$$

- $D \in R^{n \times n}$, donde los elementos d_{ij} de D se definen como:

$$d_{ij} = \begin{cases} \text{longitud del camino mínimo orientado de } i \text{ a } j & \text{si existe alguno} \\ \infty & \text{si no} \end{cases}$$

D es llamada matriz de distancias de G .

Algoritmos matriciales



$L =$

	1	2	3	4
1	0	3	∞	3
2	2	0	2	2
3	-2	∞	0	1
4	∞	4	4	0

Algoritmo de Floyd (1962)



Robert Floyd (1936–2001)

Algoritmo de Floyd (1962)

Llamamos $v_1, \dots, v_n$ a los nodos de G . El algoritmo de Floyd se basa en lo siguiente:

Algoritmo de Floyd (1962)

Llamamos $v_1, \dots, v_n$ a los nodos de G . El algoritmo de Floyd se basa en lo siguiente:

1. Si $L^0 = L$ y calculamos L^1 como

$$\ell_{ij}^1 = \min(\ell_{ij}^0, \ell_{i1}^0 + \ell_{1j}^0)$$

ℓ_{ij}^1 es la longitud de un camino mínimo de i a j con nodo intermedio v_1 o directo si $\ell_{ij}^1 < \infty$.

Algoritmo de Floyd (1962)

Llamamos $v_1, \dots, v_n$ a los nodos de G . El algoritmo de Floyd se basa en lo siguiente:

1. Si $L^0 = L$ y calculamos L^1 como

$$\ell_{ij}^1 = \min(\ell_{ij}^0, \ell_{i1}^0 + \ell_{1j}^0)$$

ℓ_{ij}^1 es la longitud de un camino mínimo de i a j con nodo intermedio v_1 o directo si $\ell_{ij}^1 < \infty$.

2. Si calculamos L^k a partir de L^{k-1} como

$$\ell_{ij}^k = \min(\ell_{ij}^{k-1}, \ell_{ik}^{k-1} + \ell_{kj}^{k-1})$$

ℓ_{ij}^k es la longitud de un camino mínimo de i a j cuyos nodos intermedios están en $\{v_1, \dots, v_k\}$ si $\ell_{ij}^k < \infty$.

Algoritmo de Floyd (1962)

Llamamos $v_1, \dots, v_n$ a los nodos de G . El algoritmo de Floyd se basa en lo siguiente:

1. Si $L^0 = L$ y calculamos L^1 como

$$\ell_{ij}^1 = \min(\ell_{ij}^0, \ell_{i1}^0 + \ell_{1j}^0)$$

ℓ_{ij}^1 es la longitud de un camino mínimo de i a j con nodo intermedio v_1 o directo si $\ell_{ij}^1 < \infty$.

2. Si calculamos L^k a partir de L^{k-1} como

$$\ell_{ij}^k = \min(\ell_{ij}^{k-1}, \ell_{ik}^{k-1} + \ell_{kj}^{k-1})$$

ℓ_{ij}^k es la longitud de un camino mínimo de i a j cuyos nodos intermedios están en $\{v_1, \dots, v_k\}$ si $\ell_{ij}^k < \infty$.

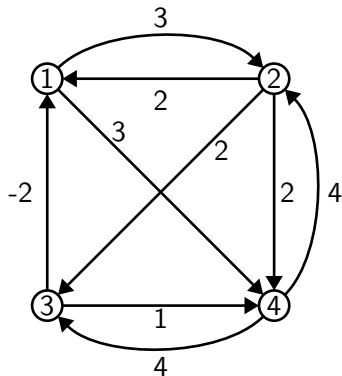
3. $D = L^n$

Algoritmo de Floyd (1962)

Asumimos que el grafo es orientado y que no hay ciclos de longitud negativa.

```
 $L^0 := L$   
para  $k$  desde 1 a  $n$  hacer  
  para  $i$  desde 1 a  $n$  hacer  
    para  $j$  desde 1 a  $n$  hacer  
       $\ell_{ij}^k := \min(\ell_{ij}^{k-1}, \ell_{ik}^{k-1} + \ell_{kj}^{k-1})$   
    fin para  
  fin para  
fin para  
retornar  $L^n$ 
```

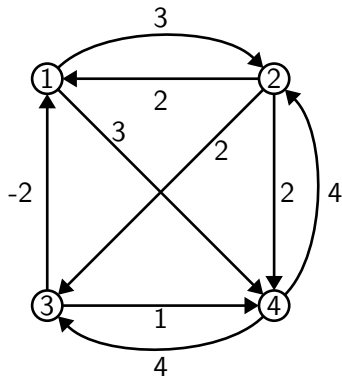
Algoritmo de Floyd (1962) - Ejemplo



$L^0 =$

	1	2	3	4
1	0	3	∞	3
2	2	0	2	2
3	-2	∞	0	1
4	∞	4	4	0

Algoritmo de Floyd (1962) - Ejemplo



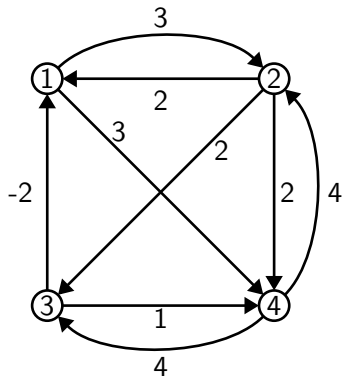
$L^0 =$

	1	2	3	4
1	0	3	∞	3
2	2	0	2	2
3	-2	∞	0	1
4	∞	4	4	0

$L^1 =$

	1	2	3	4
1				
2				
3				
4				

Algoritmo de Floyd (1962) - Ejemplo



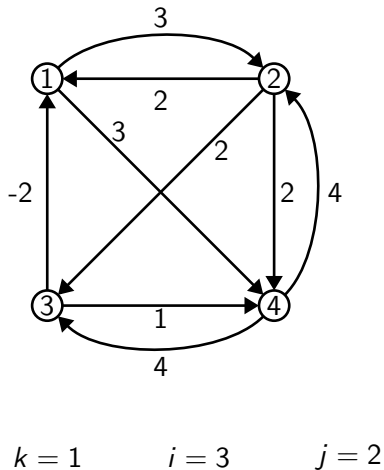
$L^0 =$

	1	2	3	4
1	0	3	∞	3
2	2	0	2	2
3	-2	∞	0	1
4	∞	4	4	0

$L^1 =$

	1	2	3	4
1	0	3	∞	3
2	2	0	2	2
3	-2			
4				

Algoritmo de Floyd (1962) - Ejemplo



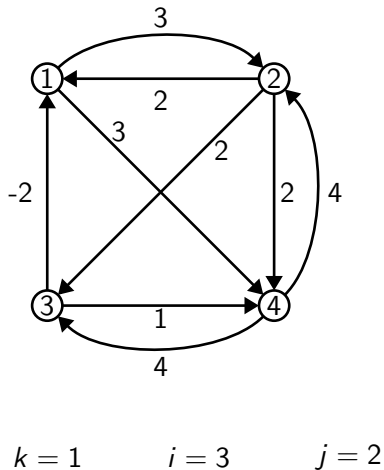
$L^0 =$

	1	2	3	4
1	0	3	∞	3
2	2	0	2	2
3	-2	∞	0	1
4	∞	4	4	0

$L^1 =$

	1	2	3	4
1	0	3	∞	3
2	2	0	2	2
3	-2			
4				

Algoritmo de Floyd (1962) - Ejemplo



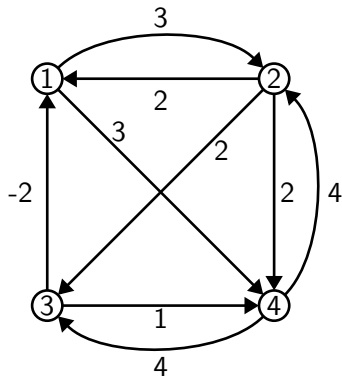
$L^0 =$

	1	2	3	4
1	0	3	∞	3
2	2	0	2	2
3	-2	∞	0	1
4	∞	4	4	0

$L^1 =$

	1	2	3	4
1	0	3	∞	3
2	2	0	2	2
3	-2	1		
4				

Algoritmo de Floyd (1962) - Ejemplo



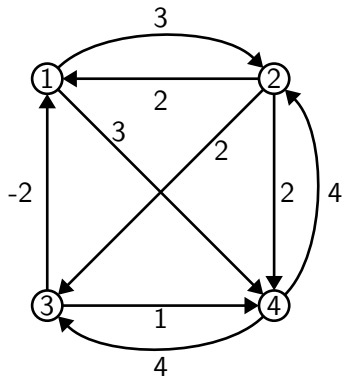
$L^0 =$

	1	2	3	4
1	0	3	∞	3
2	2	0	2	2
3	-2	∞	0	1
4	∞	4	4	0

$L^1 =$

	1	2	3	4
1	0	3	∞	3
2	2	0	2	2
3	-2	1	0	1
4	∞	4	4	0

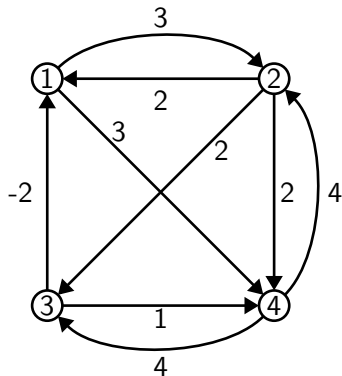
Algoritmo de Floyd (1962) - Ejemplo



$L^1 =$

	1	2	3	4
1	0	3	∞	3
2	2	0	2	2
3	-2	1	0	1
4	∞	4	4	0

Algoritmo de Floyd (1962) - Ejemplo



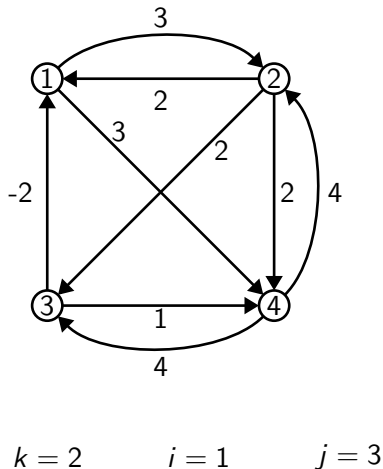
$L^1 =$

	1	2	3	4
1	0	3	∞	3
2	2	0	2	2
3	-2	1	0	1
4	∞	4	4	0

$L^2 =$

	1	2	3	4
1	0	3		
2				
3				
4				

Algoritmo de Floyd (1962) - Ejemplo



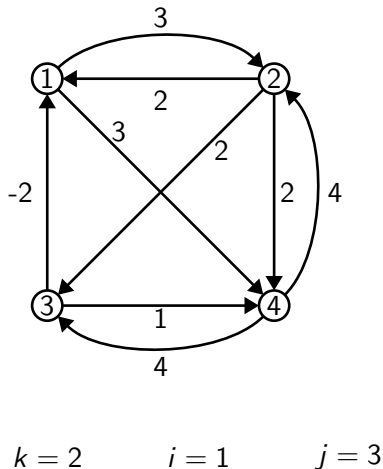
$L^1 =$

	1	2	3	4
1	0	3	∞	3
2	2	0	2	2
3	-2	1	0	1
4	∞	4	4	0

$L^2 =$

	1	2	3	4
1	0	3		
2				
3				
4				

Algoritmo de Floyd (1962) - Ejemplo



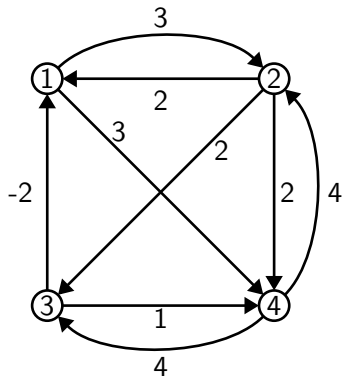
$L^1 =$

	1	2	3	4
1	0	3	∞	3
2	2	0	2	2
3	-2	1	0	1
4	∞	4	4	0

$L^2 =$

	1	2	3	4
1	0	3	5	
2				
3				
4				

Algoritmo de Floyd (1962) - Ejemplo



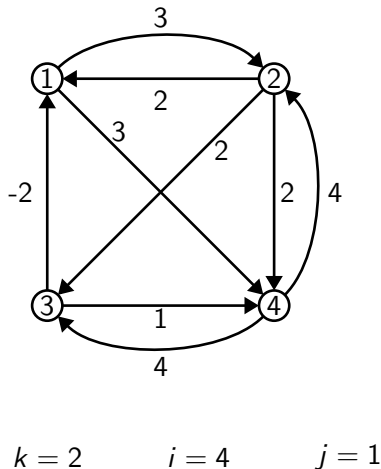
$L^1 =$

	1	2	3	4
1	0	3	∞	3
2	2	0	2	2
3	-2	1	0	1
4	∞	4	4	0

$L^2 =$

	1	2	3	4
1	0	3	5	3
2	2	0	2	2
3	-2	1	0	1
4				

Algoritmo de Floyd (1962) - Ejemplo



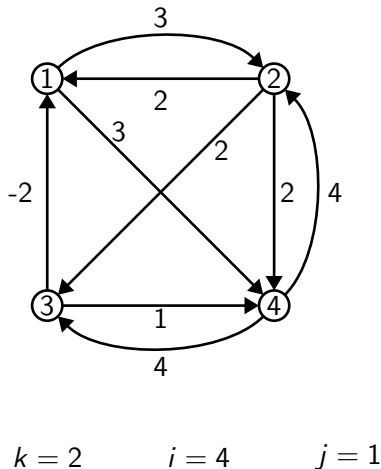
$L^1 =$

	1	2	3	4
1	0	3	∞	3
2	2	0	2	2
3	-2	1	0	1
4	∞	4	4	0

$L^2 =$

	1	2	3	4
1	0	3	5	3
2	2	0	2	2
3	-2	1	0	1
4				

Algoritmo de Floyd (1962) - Ejemplo



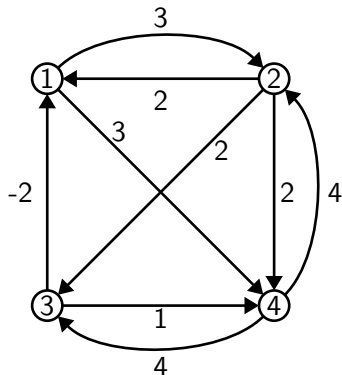
$L^1 =$

	1	2	3	4
1	0	3	∞	3
2	2	0	2	2
3	-2	1	0	1
4	∞	4	4	0

$L^2 =$

	1	2	3	4
1	0	3	5	3
2	2	0	2	2
3	-2	1	0	1
4	6			

Algoritmo de Floyd (1962) - Ejemplo



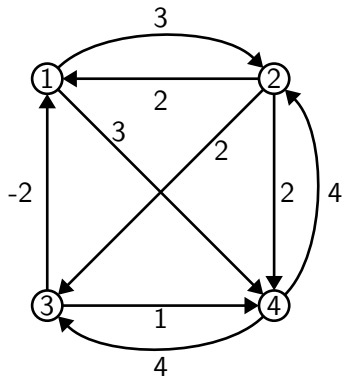
$L^1 =$

	1	2	3	4
1	0	3	∞	3
2	2	0	2	2
3	-2	1	0	1
4	∞	4	4	0

$L^2 =$

	1	2	3	4
1	0	3	5	3
2	2	0	2	2
3	-2	1	0	1
4	6	4	4	0

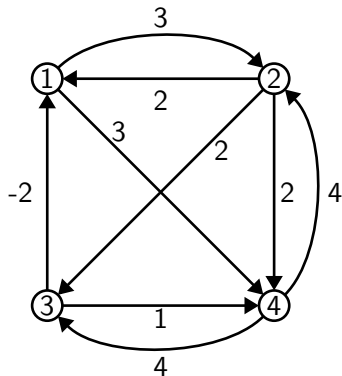
Algoritmo de Floyd (1962) - Ejemplo



$L^2 =$

	1	2	3	4
1	0	3	5	3
2	2	0	2	2
3	-2	1	0	1
4	6	4	4	0

Algoritmo de Floyd (1962) - Ejemplo



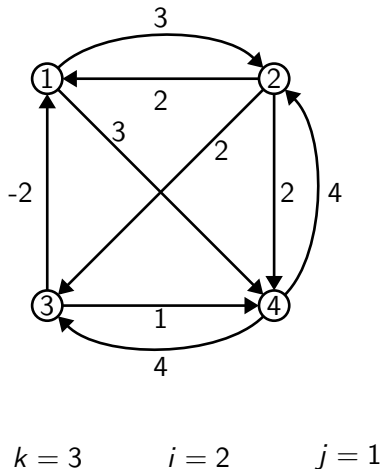
$L^2 =$

	1	2	3	4
1	0	3	5	3
2	2	0	2	2
3	-2	1	0	1
4	6	4	4	0

$L^3 =$

	1	2	3	4
1	0	3	5	3
2				
3				
4				

Algoritmo de Floyd (1962) - Ejemplo



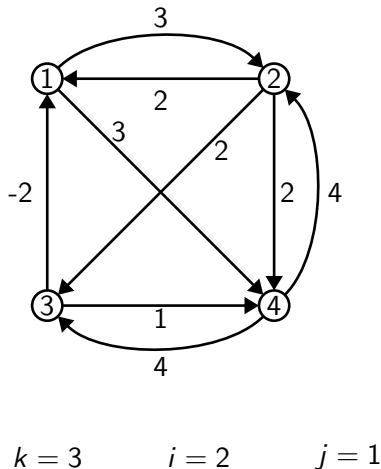
$L^2 =$

	1	2	3	4
1	0	3	5	3
2	2	0	2	2
3	-2	1	0	1
4	6	4	4	0

$L^3 =$

	1	2	3	4
1	0	3	5	3
2				
3				
4				

Algoritmo de Floyd (1962) - Ejemplo



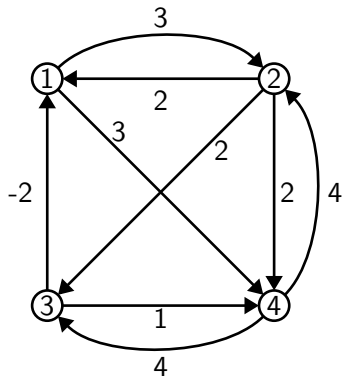
$L^2 =$

	1	2	3	4
1	0	3	5	3
2	2	0	2	2
3	-2	1	0	1
4	6	4	4	0

$L^3 =$

	1	2	3	4
1	0	3	5	3
2	0			
3				
4				

Algoritmo de Floyd (1962) - Ejemplo



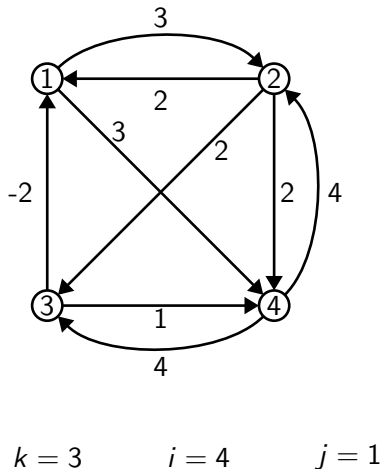
$L^2 =$

	1	2	3	4
1	0	3	5	3
2	2	0	2	2
3	-2	1	0	1
4	6	4	4	0

$L^3 =$

	1	2	3	4
1	0	3	5	3
2	0	0	2	2
3	-2	1	0	1
4				

Algoritmo de Floyd (1962) - Ejemplo



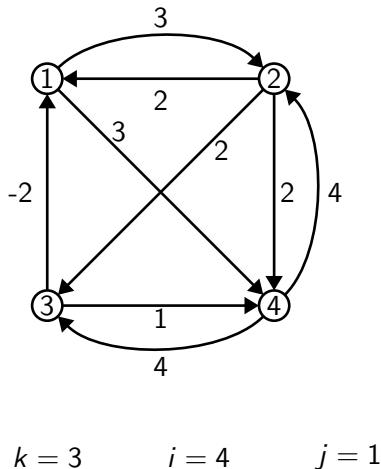
$L^2 =$

	1	2	3	4
1	0	3	5	3
2	2	0	2	2
3	-2	1	0	1
4	6	4	4	0

$L^3 =$

	1	2	3	4
1	0	3	5	3
2	0	0	2	2
3	-2	1	0	1
4				

Algoritmo de Floyd (1962) - Ejemplo



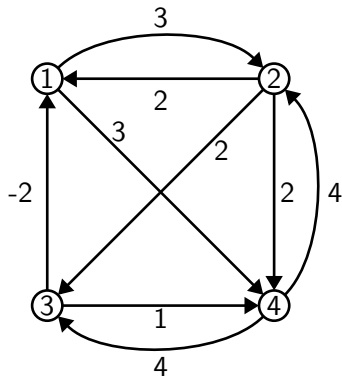
$L^2 =$

	1	2	3	4
1	0	3	5	3
2	2	0	2	2
3	-2	1	0	1
4	6	4	4	0

$L^3 =$

	1	2	3	4
1	0	3	5	3
2	0	0	2	2
3	-2	1	0	1
4	2			

Algoritmo de Floyd (1962) - Ejemplo



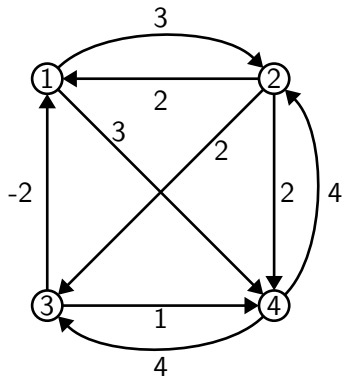
$L^2 =$

	1	2	3	4
1	0	3	5	3
2	2	0	2	2
3	-2	1	0	1
4	6	4	4	0

$L^3 =$

	1	2	3	4
1	0	3	5	3
2	0	0	2	2
3	-2	1	0	1
4	2	4	4	0

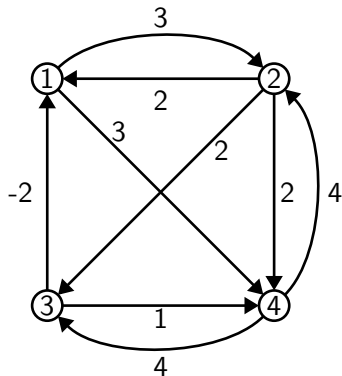
Algoritmo de Floyd (1962) - Ejemplo



$L^3 =$

	1	2	3	4
1	0	3	5	3
2	0	0	2	2
3	-2	1	0	1
4	2	4	4	0

Algoritmo de Floyd (1962) - Ejemplo



$$L^3 =$$

	1	2	3	4
1	0	3	5	3
2	0	0	2	2
3	-2	1	0	1
4	2	4	4	0

$$D = L^4 =$$

	1	2	3	4
1	0	3	5	3
2	0	0	2	2
3	-2	1	0	1
4	2	4	4	0

Algoritmo de Floyd (1962)

Lema: Al finalizar la iteración k del algoritmo de Floyd, ℓ_{ij} es la longitud de los caminos mínimos desde v_i a v_j cuyos nodos intermedios son elementos de $V_k = \{v_1, \dots, v_k\}$, si no existe ciclo de longitud negativa con todos sus vértices en V_k .

Algoritmo de Floyd (1962)

Lema: Al finalizar la iteración k del algoritmo de Floyd, ℓ_{ij} es la longitud de los caminos mínimos desde v_i a v_j cuyos nodos intermedios son elementos de $V_k = \{v_1, \dots, v_k\}$, si no existe ciclo de longitud negativa con todos sus vértices en V_k .

Teorema: El algoritmo de Floyd determina los caminos mínimos entre todos los pares de nodos de un grafo orientado sin ciclos de longitud negativa.

Algoritmo de Floyd (1962)

Lema: Al finalizar la iteración k del algoritmo de Floyd, ℓ_{ij} es la longitud de los caminos mínimos desde v_i a v_j cuyos nodos intermedios son elementos de $V_k = \{v_1, \dots, v_k\}$, si no existe ciclo de longitud negativa con todos sus vértices en V_k .

Teorema: El algoritmo de Floyd determina los caminos mínimos entre todos los pares de nodos de un grafo orientado sin ciclos de longitud negativa.

- ¿Cuál es la complejidad de algoritmo de Floyd?

Algoritmo de Floyd (1962)

Lema: Al finalizar la iteración k del algoritmo de Floyd, ℓ_{ij} es la longitud de los caminos mínimos desde v_i a v_j cuyos nodos intermedios son elementos de $V_k = \{v_1, \dots, v_k\}$, si no existe ciclo de longitud negativa con todos sus vértices en V_k .

Teorema: El algoritmo de Floyd determina los caminos mínimos entre todos los pares de nodos de un grafo orientado sin ciclos de longitud negativa.

- ▶ ¿Cuál es la complejidad de algoritmo de Floyd?
- ▶ ¿Cuánta memoria requiere?

Algoritmo de Floyd (1962)

Lema: Al finalizar la iteración k del algoritmo de Floyd, ℓ_{ij} es la longitud de los caminos mínimos desde v_i a v_j cuyos nodos intermedios son elementos de $V_k = \{v_1, \dots, v_k\}$, si no existe ciclo de longitud negativa con todos sus vértices en V_k .

Teorema: El algoritmo de Floyd determina los caminos mínimos entre todos los pares de nodos de un grafo orientado sin ciclos de longitud negativa.

- ▶ ¿Cuál es la complejidad de algoritmo de Floyd?
- ▶ ¿Cuánta memoria requiere?
- ▶ ¿Cómo podemos hacer si además de las longitudes queremos determinar los caminos mínimos?

Algoritmo de Floyd (1962)

Lema: Al finalizar la iteración k del algoritmo de Floyd, ℓ_{ij} es la longitud de los caminos mínimos desde v_i a v_j cuyos nodos intermedios son elementos de $V_k = \{v_1, \dots, v_k\}$, si no existe ciclo de longitud negativa con todos sus vértices en V_k .

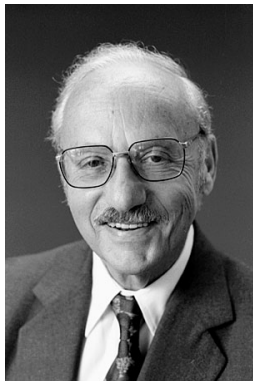
Teorema: El algoritmo de Floyd determina los caminos mínimos entre todos los pares de nodos de un grafo orientado sin ciclos de longitud negativa.

- ▶ ¿Cuál es la complejidad de algoritmo de Floyd?
- ▶ ¿Cuánta memoria requiere?
- ▶ ¿Cómo podemos hacer si además de las longitudes queremos determinar los caminos mínimos?
- ▶ ¿Cómo se puede adaptar para detectar si el grafo tiene ciclos de longitud negativa?

Algoritmo de Floyd (1962)

```
 $L^0 := L$   
para  $k$  desde 1 a  $n$  hacer  
  para  $i$  desde 1 a  $n$  hacer  
    si  $\ell_{ik}^{k-1} \neq \infty$  entonces  
      si  $\ell_{ik}^{k-1} + \ell_{ki}^{k-1} < 0$  entonces  
        retornar "Hay ciclos negativos."  
      fin si  
    para  $j$  desde 1 a  $n$  hacer  
       $\ell_{ij}^k := \min(\ell_{ij}^{k-1}, \ell_{ik}^{k-1} + \ell_{kj}^{k-1})$   
    fin para  
  fin si  
fin para  
retornar  $L$ 
```

Algoritmo de Dantzig (1966)



George Dantzig (1914–2005)

Algoritmo de Dantzig (1966)

Al finalizar la iteración $k - 1$, el algoritmo de Dantzig genera una matriz de $k \times k$ de caminos mínimos en el subgrafo inducido por los vértices $\{v_1, \dots, v_k\}$.

Algoritmo de Dantzig (1966)

Al finalizar la iteración $k - 1$, el algoritmo de Dantzig genera una matriz de $k \times k$ de caminos mínimos en el subgrafo inducido por los vértices $\{v_1, \dots, v_k\}$.

Calcula la matriz L^{k+1} a partir de la matriz L^k para $1 \leq i, j \leq k$ como:

- ▶ $L_{i,k+1}^{k+1} = \min_{1 \leq j \leq k} (L_{i,j}^k + L_{j,k+1}^k)$
- ▶ $L_{k+1,i}^{k+1} = \min_{1 \leq j \leq k} (L_{k+1,j}^k + L_{j,i}^k)$
- ▶ $L_{i,j}^{k+1} = \min(L_{i,j}^k, L_{i,k+1}^k + L_{k+1,j}^k)$

Algoritmo de Dantzig (1966)

Al finalizar la iteración $k - 1$, el algoritmo de Dantzig genera una matriz de $k \times k$ de caminos mínimos en el subgrafo inducido por los vértices $\{v_1, \dots, v_k\}$.

Calcula la matriz L^{k+1} a partir de la matriz L^k para $1 \leq i, j \leq k$ como:

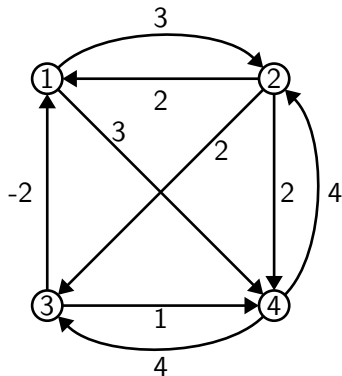
- ▶ $L_{i,k+1}^{k+1} = \min_{1 \leq j \leq k} (L_{i,j}^k + L_{j,k+1}^k)$
- ▶ $L_{k+1,i}^{k+1} = \min_{1 \leq j \leq k} (L_{k+1,j}^k + L_{j,i}^k)$
- ▶ $L_{i,j}^{k+1} = \min(L_{i,j}^k, L_{i,k+1}^k + L_{k+1,j}^k)$

Asumimos que el grafo es orientado. Detecta si hay ciclos de longitud negativa.

Algoritmo de Dantzig (1966)

```
para  $k$  desde 1 a  $n - 1$  hacer
  para  $i$  desde 1 a  $k$  hacer
     $L_{i,k+1} := \min_{1 \leq j \leq k} (L_{i,j} + L_{j,k+1})$ 
     $L_{k+1,i} := \min_{1 \leq j \leq k} (L_{k+1,j} + L_{j,i})$ 
  fin para
   $t := \min_{1 \leq i \leq k} (L_{k+1,i} + L_{i,k+1})$ 
  si  $t < 0$  entonces
    retornar "Hay ciclos de longitud negativa"
  fin si
  para  $i$  desde 1 a  $k$  hacer
    para  $j$  desde 1 a  $k$  hacer
       $L_{i,j} := \min(L_{i,j}, L_{i,k+1} + L_{k+1,j})$ 
    fin para
  fin para
fin para
retornar  $L$ 
```

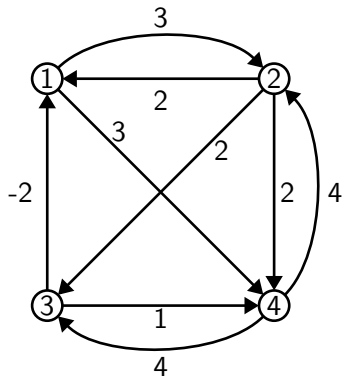
Algoritmo de Dantzig (1966) - Ejemplo



$L =$

	1	2	3	4
1	0	3	∞	3
2	2	0	2	2
3	-2	∞	0	1
4	∞	4	4	0

Algoritmo de Dantzig (1966) - Ejemplo

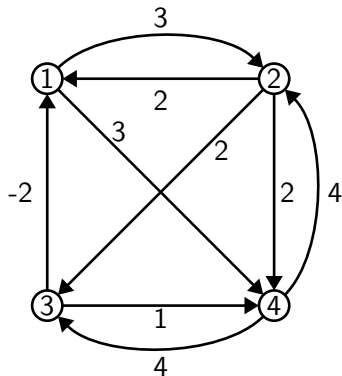


$k = 1$

$L =$

	1	2	3	4
1	0	3	∞	3
2	2	0	2	2
3	-2	∞	0	1
4	∞	4	4	0

Algoritmo de Dantzig (1966) - Ejemplo

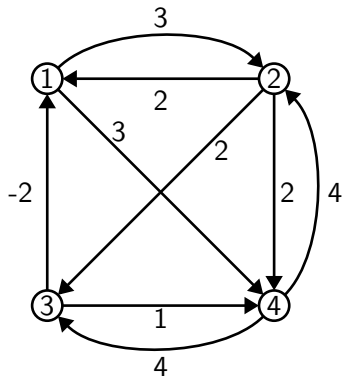


$k = 1$

$L =$

	1	2	3	4
1	0	3	∞	3
2	2	0	2	2
3	-2	∞	0	1
4	∞	4	4	0

Algoritmo de Dantzig (1966) - Ejemplo

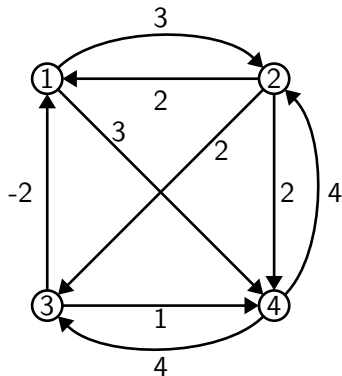


$k = 1$

$L =$

	1	2	3	4
1	0	3	∞	3
2	2	0	2	2
3	-2	∞	0	1
4	∞	4	4	0

Algoritmo de Dantzig (1966) - Ejemplo

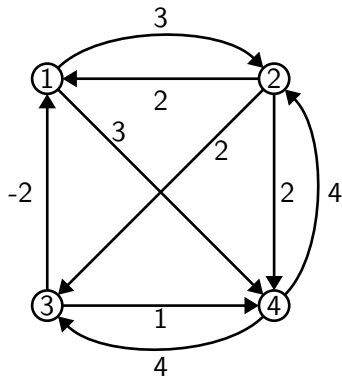


$k = 2$

$L =$

	1	2	3	4
1	0	3	∞	3
2	2	0	2	2
3	-2	∞	0	1
4	∞	4	4	0

Algoritmo de Dantzig (1966) - Ejemplo

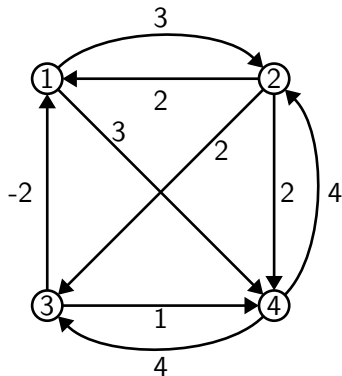


$k = 2$

$L =$

	1	2	3	4
1	0	3	∞	3
2	2	0	2	2
3	-2	∞	0	1
4	∞	4	4	0

Algoritmo de Dantzig (1966) - Ejemplo

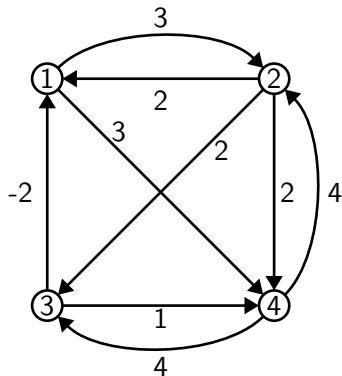


$k = 2$

$L =$

	1	2	3	4
1	0	3	5	3
2	2	0	2	2
3	-2	1	0	1
4	∞	4	4	0

Algoritmo de Dantzig (1966) - Ejemplo

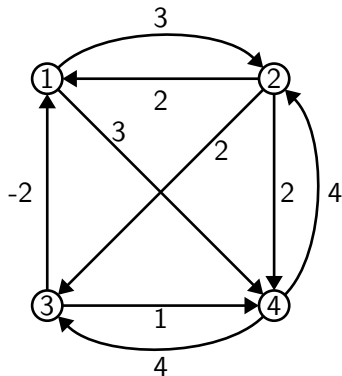


$k = 2$

$L =$

	1	2	3	4
1	0	3	5	3
2	2	0	2	2
3	-2	1	0	1
4	∞	4	4	0

Algoritmo de Dantzig (1966) - Ejemplo

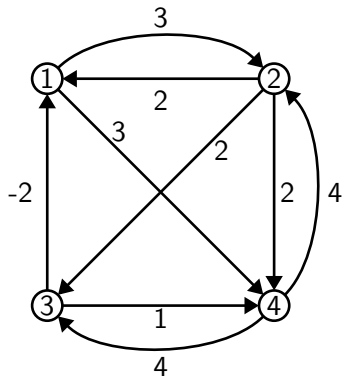


$k = 3$

$L =$

	1	2	3	4
1	0	3	5	3
2	0	0	2	2
3	-2	1	0	1
4	∞	4	4	0

Algoritmo de Dantzig (1966) - Ejemplo

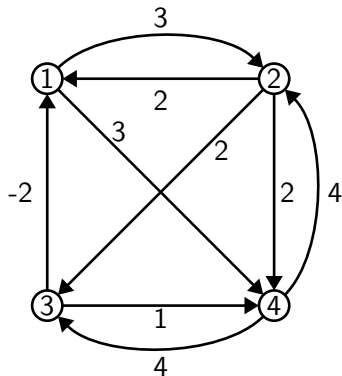


$k = 3$

$L =$

	1	2	3	4
1	0	3	5	3
2	0	0	2	2
3	-2	1	0	1
4	∞	4	4	0

Algoritmo de Dantzig (1966) - Ejemplo

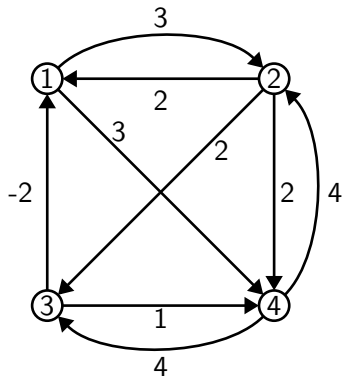


$k = 3$

$L =$

	1	2	3	4
1	0	3	5	3
2	0	0	2	2
3	-2	1	0	1
4	2	4	4	0

Algoritmo de Dantzig (1966) - Ejemplo

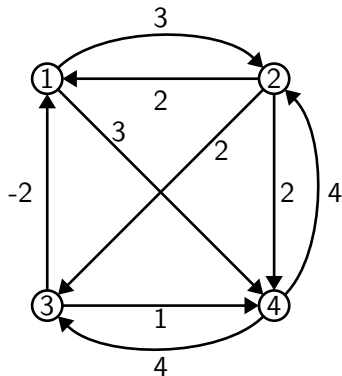


$k = 3$

$L =$

	1	2	3	4
1	0	3	5	3
2	0	0	2	2
3	-2	1	0	1
4	2	4	4	0

Algoritmo de Dantzig (1966) - Ejemplo



$k = 4$

$L =$

	1	2	3	4
1	0	3	5	3
2	0	0	2	2
3	-2	1	0	1
4	2	4	4	0

Algoritmo de Dantzig (1966)

Lema Al finalizar la iteración $k - 1$ del algoritmo de Dantzig, la matriz de $k \times k$ generada contiene la longitud de los caminos mínimos en el subgrafo inducido por los vértices $\{v_1, \dots, v_k\}$ sin ciclos de longitud negativa.

Algoritmo de Dantzig (1966)

Lema Al finalizar la iteración $k - 1$ del algoritmo de Dantzig, la matriz de $k \times k$ generada contiene la longitud de los caminos mínimos en el subgrafo inducido por los vértices $\{v_1, \dots, v_k\}$ sin ciclos de longitud negativa.

Teorema: El algoritmo de Dantzig determina los caminos mínimos entre todos los pares de nodos de un grafo orientado sin ciclos de longitud negativa.

Algoritmo de Dantzig (1966)

Lema Al finalizar la iteración $k - 1$ del algoritmo de Dantzig, la matriz de $k \times k$ generada contiene la longitud de los caminos mínimos en el subgrafo inducido por los vértices $\{v_1, \dots, v_k\}$ sin ciclos de longitud negativa.

Teorema: El algoritmo de Dantzig determina los caminos mínimos entre todos los pares de nodos de un grafo orientado sin ciclos de longitud negativa.

- ¿Cuál es la complejidad del algoritmo de Dantzig?

Algoritmo de Dantzig (1966)

Lema Al finalizar la iteración $k - 1$ del algoritmo de Dantzig, la matriz de $k \times k$ generada contiene la longitud de los caminos mínimos en el subgrafo inducido por los vértices $\{v_1, \dots, v_k\}$ sin ciclos de longitud negativa.

Teorema: El algoritmo de Dantzig determina los caminos mínimos entre todos los pares de nodos de un grafo orientado sin ciclos de longitud negativa.

- ▶ ¿Cuál es la complejidad del algoritmo de Dantzig?
- ▶ ¿Qué diferencia tiene con el algoritmo de Floyd?

Algoritmo de Dantzig (1966)

Lema Al finalizar la iteración $k - 1$ del algoritmo de Dantzig, la matriz de $k \times k$ generada contiene la longitud de los caminos mínimos en el subgrafo inducido por los vértices $\{v_1, \dots, v_k\}$ sin ciclos de longitud negativa.

Teorema: El algoritmo de Dantzig determina los caminos mínimos entre todos los pares de nodos de un grafo orientado sin ciclos de longitud negativa.

- ▶ ¿Cuál es la complejidad del algoritmo de Dantzig?
- ▶ ¿Qué diferencia tiene con el algoritmo de Floyd?
- ▶ ¿Qué ventajas o desventajas tiene?